1. IMPORTS

In [87]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer

from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder

from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from xgboost import XGBClassifier

from sklearn.metrics import (
    r2_score, mean_absolute_error, mean_squared_error, roc_auc_score, classifica
    confusion_matrix, roc_curve, auc, precision_recall_curve, accuracy_score, re
)

import statsmodels.api as sm
from statsmodels.stats.outliers_influence import variance_inflation_factor
import statsmodels.formula.api as smf

import shap

import warnings
warnings.filterwarnings("ignore")

pd.options.display.float_format = '{:,.2f}'.format

pd.set_option('display.max_columns', None)  # Mostra todas as colunas
```

2. LOAD DATA

In [88]:
```
df = pd.read_csv("./data/acquisition_train.csv")
```

In [89]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 45000 entries, 0 to 44999
Data columns (total 43 columns):
 #   Column                                          Non-Null Count  Dtype
---  ------                                          --------------  -----
 0   ids                                             45000 non-null  str
 1   target_default                                  41741 non-null  object
 2   score_1                                         44438 non-null  str
 3   score_2                                         44438 non-null  str
 4   score_3                                         44438 non-null  float64
 5   score_4                                         45000 non-null  float64
 6   score_5                                         45000 non-null  float64
 7   score_6                                         45000 non-null  float64
 8   risk_rate                                       44438 non-null  float64
 9   last_amount_borrowed                            15044 non-null  float64
 10  last_borrowed_in_months                         15044 non-null  float64
 11  credit_limit                                    31200 non-null  float64
 12  reason                                          44434 non-null  str
 13  income                                          44438 non-null  float64
 14  facebook_profile                                40542 non-null  object
 15  state                                           44438 non-null  str
 16  zip                                             44438 non-null  str
 17  channel                                         44438 non-null  str
 18  job_name                                        41664 non-null  str
 19  real_state                                      44438 non-null  str
 20  ok_since                                        18455 non-null  float64
 21  n_bankruptcies                                  44303 non-null  float64
 22  n_defaulted_loans                               44426 non-null  float64
 23  n_accounts                                      44438 non-null  float64
 24  n_issues                                        33456 non-null  float64
 25  application_time_applied                        45000 non-null  str
 26  application_time_in_funnel                      45000 non-null  int64
 27  email                                           45000 non-null  str
 28  external_data_provider_credit_checks_last_2_year  22372 non-null  float64
 29  external_data_provider_credit_checks_last_month   45000 non-null  int64
 30  external_data_provider_credit_checks_last_year    29876 non-null  float64
 31  external_data_provider_email_seen_before        42767 non-null  float64
 32  external_data_provider_first_name               45000 non-null  str
 33  external_data_provider_fraud_score              45000 non-null  int64
 34  lat_lon                                         43637 non-null  str
 35  marketing_channel                               41422 non-null  str
 36  profile_phone_number                            45000 non-null  str
 37  reported_income                                 45000 non-null  float64
 38  shipping_state                                  45000 non-null  str
 39  shipping_zip_code                               45000 non-null  int64
 40  profile_tags                                    45000 non-null  str
 41  user_agent                                      44278 non-null  str
 42  target_fraud                                    1522 non-null   str
dtypes: float64(18), int64(4), object(2), str(19)
memory usage: 14.8+ MB
```

DROP NA ONLY target_default

In [90]:
```
target = "target_default"
df2 = df.copy()
df2[target] = pd.to_numeric(df2[target], errors='coerce')
df2 = df2.dropna(subset=[target])
```

3. CLEANING

In [91]:
```
drop_cols = ["ids","score_1","score_2","zip","shipping_zip_code","lat_lon","emai
             "application_time_in_funnel", "user_agent","external_data_provider_
             "external_data_provider_fraud_score", "profile_tags", "profile_phon
             "shipping_state", "target_fraud", "marketing_channel", "reason", "s
             "job_name", "real_state", "facebook_profile","external_data_provide

df2 = df2.drop(columns = [c for c in drop_cols if c in df2.columns])

if "reported_income" in df2.columns:
    df2 = df2[df2["reported_income"] < 1e7]
```

In [92]:
```
df2.shape
```

Out[92]:
```
(41480, 20)
```

4. FEATURE ENGINEERING

In [93]:
```
if all(c in df2.columns for c in ['last_amount_borrowed','income']):
    df2['debt_to_income'] = df2['last_amount_borrowed'] / (df2['income'] + 1)

if all(c in df2.columns for c in ['last_amount_borrowed','credit_limit']):
    df2['credit_utilization'] = df2['last_amount_borrowed'] / (df2['credit_limit

neg_cols = ['n_bankruptcies','n_defaulted_loans','n_issues']
if all(c in df2.columns for c in neg_cols):
    df2['total_negatives'] = df2[neg_cols].sum(axis=1)

if 'income' in df2.columns:
    df2['income_log'] = np.log1p(df2['income'])

score_cols = [c for c in df2.columns if 'score_' in c]
for col in score_cols:
    df2[col] = pd.to_numeric(df2[col], errors = "coerce")
if score_cols:
    df2['score_mean'] = df2[score_cols].mean(axis=1)

df2[score_cols].dtypes
```

Out[93]:
```
score_3    float64
score_4    float64
score_5    float64
score_6    float64
dtype: object
```

In [94]:
```
df2 = df2.drop(columns=['score_3','score_4','score_5','score_6'])
```

In [95]:
```
df2.describe()
```

Out[95]:

	target_default	risk_rate	last_amount_borrowed	last_borrowed_in_months	credit
count	41,480.00	41,480.00	14,035.00	14,035.00	28,4
mean	0.16	0.29	13,323.05	40.59	33,8
std	0.37	0.10	7,918.26	9.44	36,1
min	0.00	0.00	1,005.18	36.00	
25%	0.00	0.22	7,210.12	36.00	9,9
50%	0.00	0.29	12,010.95	36.00	25,2
75%	0.00	0.36	18,028.97	36.00	46,5
max	1.00	0.90	35,059.60	60.00	448,2

In [96]:
```
df2.columns
```

Out[96]:
```
Index(['target_default', 'risk_rate', 'last_amount_borrowed',
       'last_borrowed_in_months', 'credit_limit', 'income', 'channel',
       'ok_since', 'n_bankruptcies', 'n_defaulted_loans', 'n_accounts',
       'n_issues', 'external_data_provider_credit_checks_last_2_year',
       'external_data_provider_credit_checks_last_month',
       'external_data_provider_credit_checks_last_year', 'reported_income',
       'debt_to_income', 'credit_utilization', 'total_negatives', 'income_log',
       'score_mean'],
      dtype='str')
```

In [97]:
```
df2.shape
```

Out[97]:
```
(41480, 21)
```

5. SPLIT

In [98]:
```
X = df2.drop(columns=[target])
y = df2[target]

X_train, X_test, y_train, y_test = train_test_split(X,y,test_size=0.3, stratify=
```

MACHINE LEARNING

6. PIPELINE

In [99]:
```
num_cols = X_train.select_dtypes(include=['int64','float64']).columns
cat_cols = X_train.select_dtypes(include=['object']).columns

numeric_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler())
])

categorical_pipeline = Pipeline([
```

```
    ('imputer', SimpleImputer(strategy='constant', fill_value='Unknown')),
    ('onehot', OneHotEncoder(handle_unknown='ignore'))
])

preprocessor = ColumnTransformer([
    ('num', numeric_pipeline, num_cols),
    ('cat', categorical_pipeline, cat_cols)
])
```

7. MODELOS

In [100…

```
scale_pos_weight = sum(y_train == 0) / sum(y_train == 1)

models = {
    'Logistic Regression': LogisticRegression(max_iter=1000, class_weight='balan
    'Random Forest': RandomForestClassifier(n_estimators=200, max_depth=8, randc
    'Gradient Boosting': GradientBoostingClassifier(),
    'XGBoost': XGBClassifier(
        n_estimators=300,
        max_depth=5,
        learning_rate=0.05,
        scale_pos_weight=scale_pos_weight,
        subsample=0.8,
        colsample_bytree=0.8,
        eval_metric='logloss',
        use_label_encoder=False
    )
}
```

8. TREINO

In [101…

```
pipelines = {}
results = []

for name, model in models.items():
    pipe = Pipeline([
        ('preprocessor', preprocessor),
        ('model', model)
    ])

    pipe.fit(X_train, y_train)
    pipelines[name] = pipe

    y_pred = pipe.predict_proba(X_test)[:,1]
    auc = roc_auc_score(y_test, y_pred)

    results.append((name, auc))
    print(f'{name}: AUC = {auc:.4f}')
```

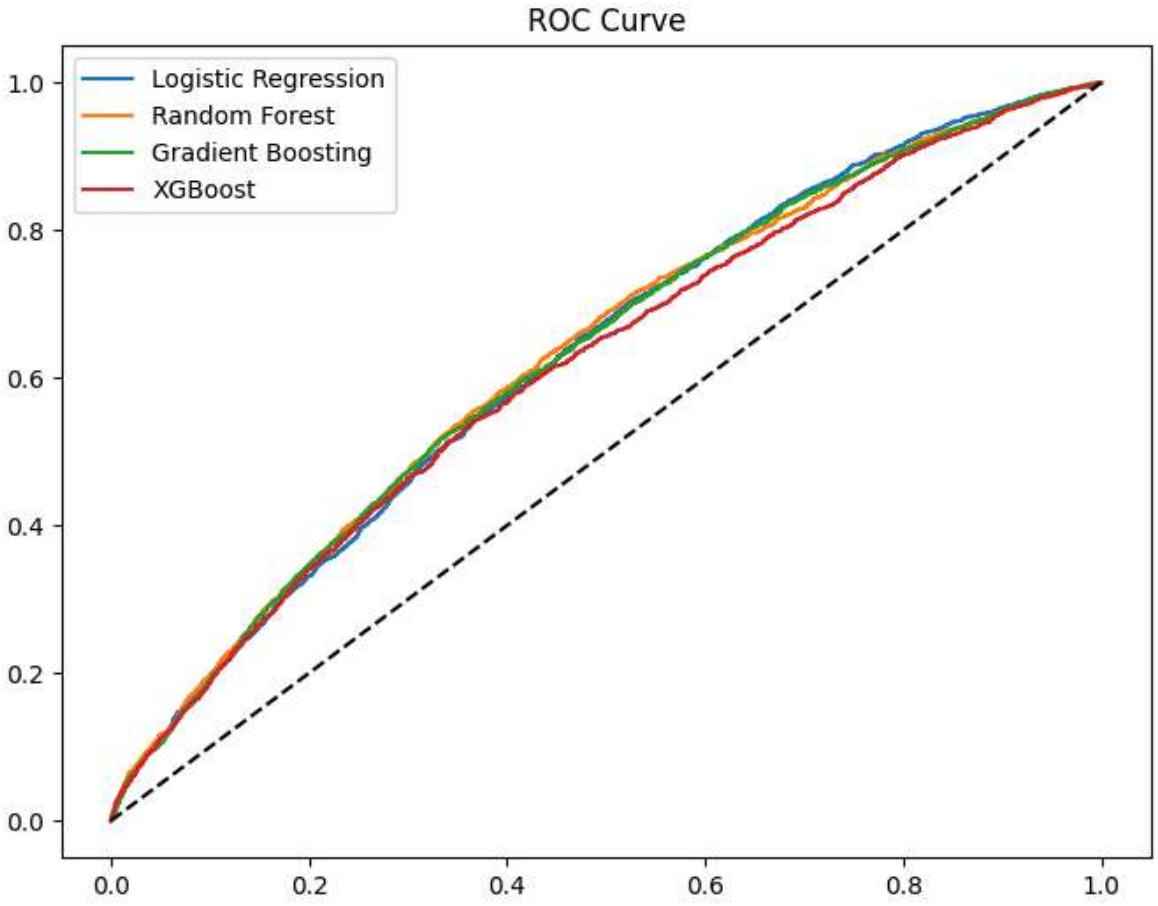
```
Logistic Regression: AUC = 0.6252
Random Forest: AUC = 0.6283
Gradient Boosting: AUC = 0.6259
XGBoost: AUC = 0.6147
```

9. ROC

In [102…
```
plt.figure(figsize=(8,6))

for name, pipe in pipelines.items():
    y_pred = pipe.predict_proba(X_test)[:,1]
    fpr, tpr, _ = roc_curve(y_test, y_pred)
    plt.plot(fpr, tpr, label=name)

plt.plot([0,1],[0,1],'k--')
plt.legend()
plt.title('ROC Curve')
plt.show()
```



10. KS

In [103…
```
def ks_stat(y_true, y_pred):
    df_ks = pd.DataFrame({'y': y_true, 'prob': y_pred}).sort_values('prob')
    df_ks['cum_good'] = (df_ks['y'] == 0).cumsum() / sum(df_ks['y'] == 0)
    df_ks['cum_bad'] = (df_ks['y'] == 1).cumsum() / sum(df_ks['y'] == 1)
    return max(abs(df_ks['cum_good'] - df_ks['cum_bad']))

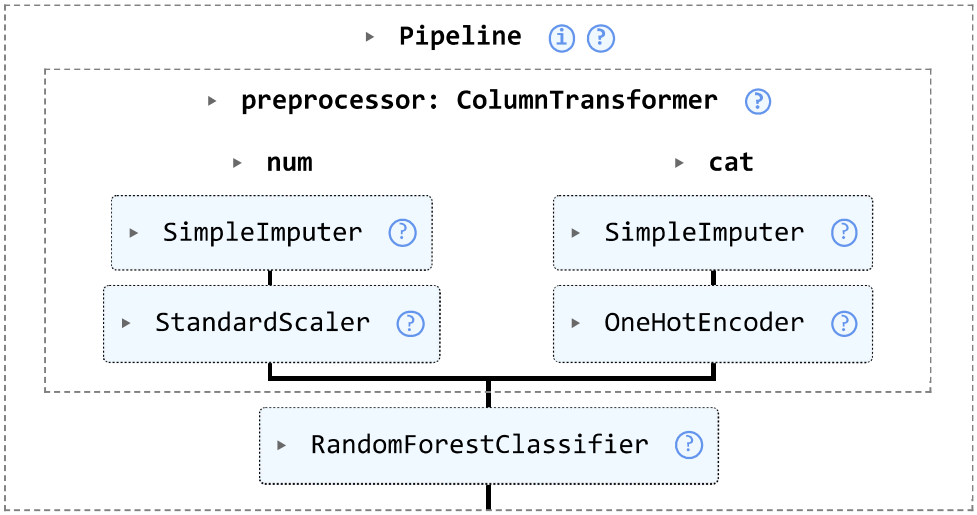
for name, pipe in pipelines.items():
    y_pred = pipe.predict_proba(X_test)[:,1]
    print(f'{name} KS:', ks_stat(y_test, y_pred))
```

```
Logistic Regression KS: 0.18071191397730185
Random Forest KS: 0.19035370369407417
Gradient Boosting KS: 0.18445584958160938
XGBoost KS: 0.17566135640244768
```

11. MELHOR MODELO

In [104…
```
best_model = sorted(results, key=lambda x: x[1], reverse=True)[0]
best_pipe = pipelines[best_model[0]]
best_pipe
```

Out[104…



12. FEATURE IMPORTANCE

In [105…
```
model = best_pipe.named_steps['model']

if hasattr(model, 'feature_importances_'):
    ohe = best_pipe.named_steps['preprocessor'] \
        .named_transformers_['cat']['onehot']

    cat_features = ohe.get_feature_names_out(cat_cols)
    all_features = list(num_cols) + list(cat_features)

    feat_imp = pd.DataFrame({
        'feature': all_features,
        'importance': model.feature_importances_
    }).sort_values(by='importance', ascending=False)

    print("\nTop Features:")
    print(feat_imp.head(10))
```

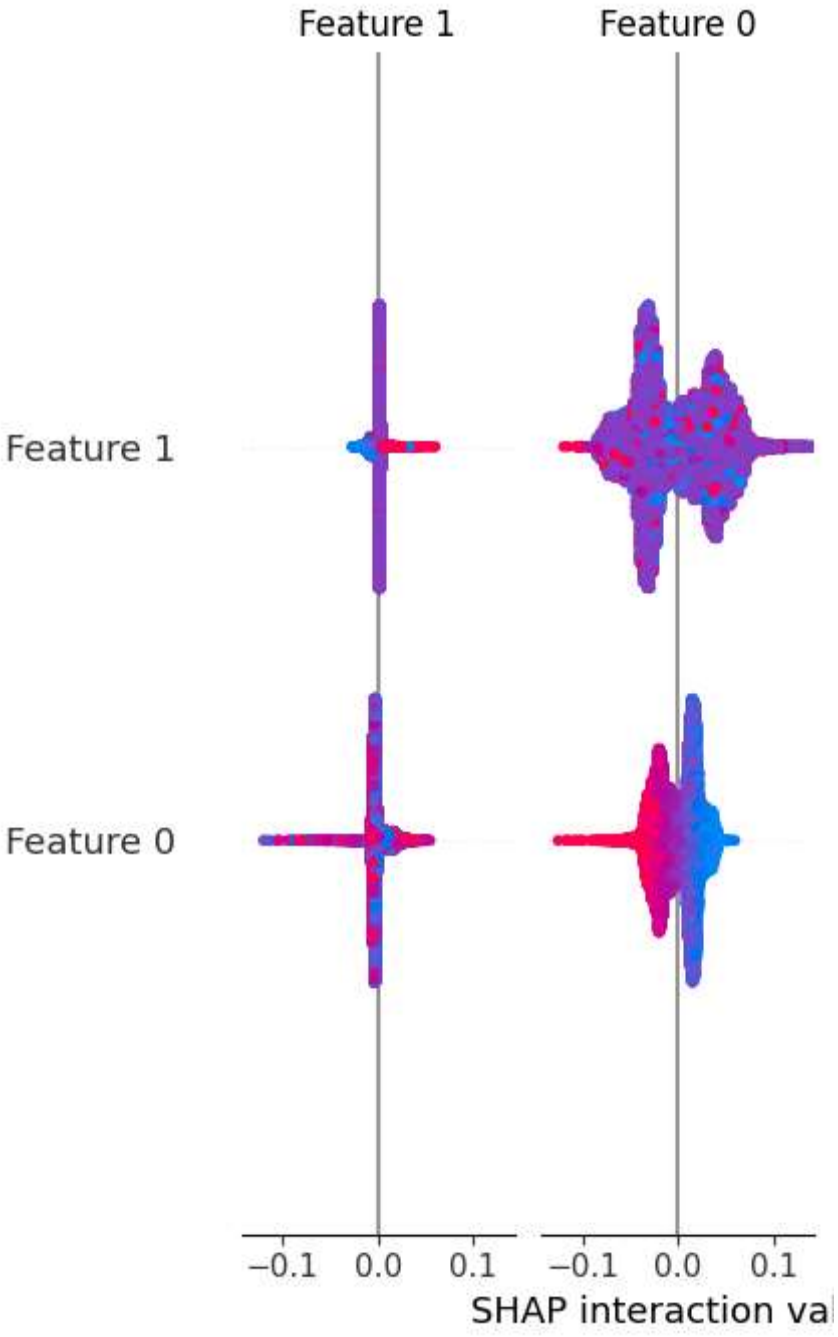
```
Top Features:
                    feature  importance
0                 risk_rate        0.14
17              income_log         0.10
2   last_borrowed_in_months        0.09
4                    income        0.09
14          debt_to_income         0.09
18               score_mean        0.08
1      last_amount_borrowed        0.07
13          reported_income        0.06
15       credit_utilization        0.05
3              credit_limit        0.05
```

13. SHAP

In [106…

```
try:
    X_processed = best_pipe.named_steps['preprocessor'].transform(X_train)
    explainer = shap.Explainer(best_pipe.named_steps['model'])
    shap_values = explainer(X_processed)

    shap.summary_plot(shap_values, X_processed)
except:
    print("SHAP não disponível para este modelo.")
```



14. SCORE

In [107…

```
y_pred = best_pipe.predict_proba(X_test)[:,1]

score = 1000 - (y_pred * 1000)

score_df = pd.DataFrame({
    'prob_default': y_pred,
```

```
    'score': score
})

print(score_df.head())
```

```
   prob_default  score
0          0.12 879.25
1          0.21 785.07
2          0.17 832.24
3          0.12 882.69
4          0.11 888.41
```

In [ ]: